

TYPES OF SQL JOINS

BSIS Database Management

1. What is a JOIN?

A **JOIN** is an SQL operation used to combine related information from two or more tables.

In a relational database, we usually do not put all information into one large table. Instead, we separate information into different tables and connect them using related columns.

Simple idea:

JOIN = Connect related data from different tables.

For example, we have two tables:

Students

student_id	name
------------	------

1	Ana
2	Ben
3	Carlo

Enrollments

student_id	subject
------------	---------

1	Database
2	Networking
4	Programmin g

The column student_id is what allows us to connect the two tables.

Notice that:

- Ana has student_id = 1
- Ben has student_id = 2

- Carlo has student_id = 3
- Enrollment 4 does not have a matching student

This difference is important because **different JOIN types decide which records will appear in the result.**

2. INNER JOIN

Meaning

An **INNER JOIN** returns only the records that have a matching value in **both tables**.

Think of it as:

"Show me only the records that match."

Example

```
SELECT Students.name, Enrollments.subject
FROM Students
INNER JOIN Enrollments
ON Students.student_id = Enrollments.student_id;
```

Result

name subject

Ana Database

Ben Networkin
g

Why is Carlo missing?

Because Carlo has student_id = 3, but there is no 3 in the Enrollments table.

Why is student ID 4 missing?

Because there is no student with student_id = 4.

Easy memory trick:

INNER JOIN = MATCH ONLY

3. LEFT JOIN

Meaning

A **LEFT JOIN** returns:

1. **All records from the LEFT table**
2. Matching records from the RIGHT table

Even if there is no match, the record from the LEFT table is still displayed.

Think of it as:

"Keep everyone from the left table."

Here:

Students = LEFT table

Enrollments = RIGHT table

Example

```
SELECT Students.name, Enrollments.subject  
FROM Students  
LEFT JOIN Enrollments  
ON Students.student_id = Enrollments.student_id;
```

Result

name subject

Ana Database

Ben Networkin
g

Carlo NULL

Why is Carlo included?

Because **LEFT JOIN keeps every record from the Students table**, even if there is no matching enrollment.

NULL means there is no corresponding value.

Easy memory trick:

LEFT JOIN = Keep everything on the LEFT

4. RIGHT JOIN

Meaning

A **RIGHT JOIN** is the opposite of a LEFT JOIN.

It returns:

1. **All records from the RIGHT table**
2. Matching records from the LEFT table

Think:

"Keep everyone from the right table."

Example

```
SELECT Students.name, Enrollments.subject
FROM Students
RIGHT JOIN Enrollments
ON Students.student_id = Enrollments.student_id;
```

Result

name subject

Ana Database

Ben Networking

NULL Programmin
g

Why is Programming included?

Because the Enrollments table is on the **RIGHT**, and RIGHT JOIN keeps all records from that table.

Enrollment student_id = 4 has no matching student, so the student information becomes NULL.

Easy memory trick:

RIGHT JOIN = Keep everything on the RIGHT

5. FULL OUTER JOIN

Meaning

A **FULL JOIN** returns **everything from both tables**.

It includes:

- Matching records
- Unmatched records from the LEFT table
- Unmatched records from the RIGHT table

Think:

"Show me everything from both sides."

Example

```
SELECT Students.name, Enrollments.subject
FROM Students
FULL OUTER JOIN Enrollments
ON Students.student_id = Enrollments.student_id;
```

Result

name subject

Ana Database

Ben Networking

Carlo NULL

NULL Programmin
g

Let's understand:

Ana + Database → Match

Ben + Networking → Match

Carlo + NULL → Student exists, but enrollment does not

NULL + Programming → Enrollment exists, but student does not

Important note

Your material points out that FULL OUTER JOIN is not directly supported by **MySQL**, although it is available in systems such as PostgreSQL and SQL Server.

Easy memory trick:

FULL JOIN = EVERYTHING FROM BOTH

6. CROSS JOIN

Meaning

A **CROSS JOIN** is different from the other JOINS.

Instead of looking for matching values, it combines **every row from the first table with every row from the second table.**

Think:

"Everyone with everyone."

Example

Suppose we have:

Students

student

t

Ana

Ben

Carlo

3 students

Subjects

subject

Database

Networking

g

2 subjects

A CROSS JOIN produces:

3 × 2 = 6 rows

Result

student	subject
----------------	----------------

Ana	Database
-----	----------

Ana	Networking
-----	------------

Ben	Database
-----	----------

Ben	Networking
-----	------------

Carlo	Database
-------	----------

Carlo	Networking
-------	------------

Every student is combined with every subject.

SQL

```
SELECT Students.name, Subjects.subject
```

```
FROM Students
```

```
CROSS JOIN Subjects;
```

Notice that a CROSS JOIN normally does **not need an ON condition**.

Easy memory trick:

CROSS JOIN = EVERYONE × EVERYONE

7. NATURAL JOIN

Meaning

A **NATURAL JOIN** automatically looks for columns that have the **same name** and compatible data types in both tables.

For example:

Employee

emp_id emp_name

101 Juan

102 Maria

103 Pedro

Department

dep_id dep_name emp_id

1 IT 101

2 HR 102

3 Finance 103

Both tables contain:

emp_id

Therefore, we can write:

```
SELECT dep_name, emp_name
```

```
FROM Department
```

```
NATURAL JOIN Employee;
```

SQL automatically uses the common emp_id column to connect the tables.

Result

dep_name emp_name

IT Juan

HR Maria

Finance Pedro

Professor's advice

NATURAL JOIN is convenient, but you should be careful.

If two tables have several columns with the same names, NATURAL JOIN may use **all of those common columns** when matching. This can produce unexpected results.

For professional database development, an explicit JOIN using ON is usually easier to understand and control.

8. What About KEY JOIN?

This is an important point from your material.

KEY JOIN is not a standard SQL JOIN keyword.

When your lesson mentions a key-based JOIN, understand it as a JOIN based on related keys, commonly:

Primary Key ↕ Foreign Key

For example:

```
SELECT Department.dep_name, Employee.emp_name
```

```
FROM Department
```

```
JOIN Employee
```

```
ON Department.emp_id = Employee.emp_id;
```

Here:

- Employee.emp_id identifies the employee.
- Department.emp_id refers to that employee.
- ON tells SQL which columns should be matched.

The important concept is:

Primary Key + Foreign Key → Relationship → JOIN → Related Data

Your material recommends focusing on the standard form:

JOIN ... ON ...

rather than treating KEY JOIN as a standard SQL command.

9. The General JOIN Syntax

The basic structure is:

FROM table1

JOIN table2

ON table1.column = table2.column;

For example:

SELECT employees.emp_name, departments.dep_name

FROM employees

INNER JOIN departments

ON employees.dep_id = departments.dep_id;

Understand each part:

FROM employees

→ Start with the employees table.

INNER JOIN departments

→ Connect the departments table.

ON employees.dep_id = departments.dep_id

→ This is the rule that tells SQL how the two tables are related.

Your material describes FROM as the starting point and the JOIN as the operation that combines related data from multiple tables.

10. The Most Important Difference

Let's put the major JOIN types side by side.

JOIN Type	What does it keep?
INNER JOIN	Matching records only
LEFT JOIN	Everything from LEFT + matches from RIGHT
RIGHT JOIN	Everything from RIGHT + matches from LEFT
FULL JOIN	Everything from BOTH tables
CROSS JOIN	Every possible combination
NATURAL JOIN	Matches automatically using common column names

This matches the quick-reference summary in your course material.

11. Easy Way to Remember

Imagine **Students** on the LEFT and **Enrollments** on the RIGHT.

INNER

Only the students who have enrollment

Match only.

LEFT

Every student, whether enrolled or not

Left side is the priority.

RIGHT

Every enrollment, whether the student exists or not

Right side is the priority.

FULL

Every student + every enrollment

Both sides are included.

CROSS

Every student paired with every enrollment

Everyone with everyone.

NATURAL

SQL automatically looks for common column names

"Find the common columns for me."

12. Professor's Practical Example

Imagine you are developing a **BSIS Student Management System**.

You have:

STUDENTS

student_id	name
-------------------	-------------

1	Ana
---	-----

2	Ben
---	-----

3	Carlo
---	-------

ENROLLMENTS

student_id	subject
-------------------	----------------

1	Database
---	----------

2	Networking
---	------------

4	Programmin g
---	-----------------

Now ask different questions.

Question 1:

"Which students have enrolled in a subject?"

Use:

INNER JOIN

Because we only want matching records.

Question 2:

"Show all students, including students who have not enrolled yet."

Use:

LEFT JOIN

Because every student must appear.

Question 3:

"Show all enrollment records, even if the student record is missing."

Use:

RIGHT JOIN

Because every enrollment must appear.

Question 4:

"Show everything from both tables."

Use:

FULL OUTER JOIN

where supported by the DBMS.

Question 5:

"Create every possible student-subject combination."

Use:

CROSS JOIN

13. Final BSIS Takeaway

As a BSIS student, do not memorize JOINS only by their names. Understand **what problem each JOIN solves**.

The most important question to ask yourself is:

"Which records do I want to keep?"

If you want:

- **Only matching data** → INNER JOIN
- **Everything from the first/left table** → LEFT JOIN
- **Everything from the second/right table** → RIGHT JOIN
- **Everything from both tables** → FULL OUTER JOIN
- **Every possible combination** → CROSS JOIN
- **Automatic matching using common column names** → NATURAL JOIN

And remember the most important practical pattern:

JOIN table2

ON table1.related_column = table2.related_column;

The **JOIN** connects the tables.

The **ON condition** tells SQL **how the tables are related**.

The **keys** provide the logical relationship between the data.

That is the foundation of JOINS in relational database management.